

CHAPTER 10

10. NestJS 기초 — 프레임워크 가 구조를 강제한다

Node.js · Express · NestJS 강의 · 2026

지난 여섯 장(04~09)에서 만든 Express 서버는 자유로웠다. 라우트를 한 파일에 몰아넣어도, 비즈니스 로직을 핸들러 안에 그대로 풀어써도, 검증을 `if` 문으로 손수 짜도 다 돌아갔다. 자유는 편하지만 사람이 많아지고 코드가 커지면 독이 된다. "이 로직은 어디에 뒀야 하나", "검증은 누가 책임지나" 같은 질문에 매번 팀마다 다른 답을 내고, 그 답이 파일마다 또 달라진다.

NestJS는 정반대 길을 택한 프레임워크다. 요청을 받는 자리(**컨트롤러**), 로직을 처리하는 자리(**서비스**), 그것들을 묶는 자리(**모듈**)를 처음부터 분리해 놓고, "여기엔 이걸 써라"를 프레임워크가 강제한다. 자유를 일부 내주는 대신, 누가 짜도 비슷한 구조가 나온다. 규모가 커질수록 이 일관성이 곧 생산성이 된다.

이 장에서는 같은 사용자·게시글 게시판을 NestJS로 다시 만든다. 저장소는 아직 메모리 배열이다. DB는 다음 장(11 TypeORM, 12 Prisma)에서 붙이고, 지금은 **NestJS의 기본 구조** 자체에 집중한다. 컨트롤러·DTO는 이후에도 거의 그대로 두고 저장 계층만 바꾸므로, 여기서 잡아둔 구조 감각이 다음 두 장의 바탕이 된다.

이 자료는 **개념 예습용**이다. 실제 수업에서 쓰는 파일 이름과 폴더 구성, 다루는 범위는 강사에 따라 다를 수 있다. 그래서 여기서는 파일이 아니라 **무엇을 배우는가**에 초점을 둔다.

이 장에서 배우는 것

- 모듈 / 컨트롤러 / 서비스 3분할 — 각자의 역할과 경계
- 의존성 주입(DI) — 생성자에 타입만 적으면 NestJS가 인스턴스를 넣는다
- 데코레이터 — `@Module` `@Controller` `@Injectable` `@Get` `@Post` `@Body` `@Param`
- DTO + class-validator + 전역 `ValidationPipe` — 검증을 선언으로 분리
- 모듈 간 협력 — `exports` / `imports` 로 다른 모듈의 서비스를 빌려 쓰기
- NestJS CLI — `nest g resource` 로 골격을 한 번에 생성

코드는 두 기능 모듈(사용자·게시글)로 나뉘고, 각 모듈은 컨트롤러·서비스·모듈·DTO 네 종류의 구성요소로 이루어진다. Express처럼 파일 하나가 독립 서버이던 구조와 다르다. 여기서는 모든 구성요소가 모여 **하나의 앱**을 이룬다.

주제	핵심
진입점	앱을 띄우고 전역 <code>ValidationPipe</code> 를 건다
루트 모듈	기능 모듈을 모아 앱을 구성한다
컨트롤러	라우팅 — 요청을 받아 서비스에 위임
서비스	인메모리 CRUD + 도메인 타입 + HTTP 예외
DTO	요청 본문의 모양과 검증 규칙
기능 모듈	컨트롤러·서비스를 묶고 경계를 긋는다
모듈 간 협력	<code>exports</code> / <code>imports</code> 로 다른 모듈의 서비스를 빌린다

핵심 개념 — 세 종류의 자리와 그것을 채우는 방식

NestJS 코드는 거의 다 세 가지 중 하나다. 먼저 이 셋의 역할을 머리에 넣어두면 나머지가 쉽게 읽힌다.

- **컨트롤러(Controller)** — 바깥(HTTP 요청)과 맞닿는 층. "어떤 경로에 어떤 메서드로 요청이 오면 무엇을 호출할 지"만 정한다. 로직은 두지 않는다. Express의 라우트 핸들러 자리다.
- **서비스(Service)** — 실제 일을 하는 층. 데이터를 만들고 찾고 지우고, 규칙을 검사한다. HTTP가 무엇인지 몰라도 되는, 순수한 로직의 자리다.
- **모듈(Module)** — 위 둘을 한 묶음으로 등록하고 경계를 긋는 층. "이 모듈은 어떤 컨트롤러·서비스를 갖고, 무엇을 밖 에 내주고, 무엇을 밖에서 빌려 오는가"를 선언한다.

이 세 자리를 코드에 표시하는 도구가 **데코레이터(decorator)** 다. 클래스나 메서드 위에 `@` 기호로 붙는

`@Controller()`, `@Injectable()`, `@Get()` 같은 표식이다. 데코레이터는 "이 클래스는 컨트롤러다", "이 메서드는 `GET /users` 를 처리한다" 같은 **메타데이터**를 클래스에 새긴다. NestJS는 앱이 뜰 때 이 표식들을 읽어 라우트를 등록하고 객체들을 연결한다. Express에서 `app.get("/users", handler)` 라고 직접 호출하던 일을, 메서드 위에 `@Get()` 을 붙이는 선언으로 바꾼 것이다.

그리고 그 객체들을 연결하는 방식이 **의존성 주입(Dependency Injection, DI)** 이다. 핵심 한 줄로 요약하면 이렇다.

필요한 것을 직접 만들지(`new`) 않고, 생성자에 타입만 적어 두면 NestJS가 만들어서 넣는다.

게시글 서비스가 사용자 서비스를 써야 한다고 하자. Express라면 `const userService = new UsersService()` 처럼 직접 생성했을 것이다. NestJS에서는 생성자 매개변수에 `userService: UsersService` 라고 타입만 적는다. 그러면 NestJS가 알아서 `UsersService` 인스턴스를 찾아 넣는다. 앱 전체에서 같은 인스턴스 하나를 공유하므로 데이터(메

모리 배열)도 자연히 한 벌로 유지된다. 누가 무엇을 필요로 하는지는 코드에 드러나지만, 그것을 어떻게 마련할지는 프레임워크가 맡는다. 이게 NestJS가 구조를 강제하는 핵심이다.

콜아웃을 활용하라 — 이 장의 코드에는 주석으로 핵심이 콕콕 박혀 있다. `// 생성자 주입(DI) - NestJS가 UserService 인스턴스를 알아서 넣어준다.` 처럼. 코드를 읽을 때 주석을 같이 읽으면 설명의 절반은 이미 그 안에 있다.

준비

```
npm install
npm run start:dev      # → http://localhost:3000 (DB 불필요)
```

`start:dev` 는 `nest start--watch` 다. 파일을 저장할 때마다 자동으로 다시 컴파일하고 서버를 재시작한다. Express 에서 파일 하나를 `node` 로 직접 실행하던 것과 다르다. NestJS는 타입스크립트로 짠 여러 파일을 빌드해 하나의 앱으로 댕춘다. 포트는 04장과 같은 `3000` 이다.

서버가 뜨면 터미널에 한 줄이 찍히고, 그 자리에서 요청을 기다린다.

[NestJS 기초] `http://localhost:3000` (users, posts) - 인메모리 저장

진입점 — 앱을 띄우고 검증을 건다

Express에 `app.listen(3000)` 이 있었듯, NestJS에도 서버를 켜는 단 하나의 진입점이 있다.

```
import { NestFactory } from "@nestjs/core";
import { ValidationPipe } from "@nestjs/common";
import { AppModule } from "../app.module";

async function bootstrap() {
  const app = await NestFactory.create(AppModule);

  // 전역 파이프 - DTO(class-validator)로 요청 본문을 검증한다.
  // whitelist: DTO에 없는 속성은 제거 / transform: 들어온 값을 DTO 타입으로 변환
  app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));

  await app.listen(3000);
  console.log("[NestJS 기초] http://localhost:3000 (users, posts) - 인메모리 저장");
}
bootstrap();
```

한 줄씩 보자.

- `NestFactory.create(AppModule)` — 루트 모듈(`AppModule`)을 받아 앱 인스턴스를 만든다. NestJS는 이 모듈을 출발점으로 그 안에 엮인 모듈·컨트롤러·서비스를 전부 따라가며 연결한다. Express의 `const app = express()`에 해당하는 자리지만, 빈 객체가 아니라 **모듈 트리 전체를 조립한 결과**가 나온다는 점이 다르다.
- `app.useGlobalPipes(new ValidationPipe(.))` — 모든 요청 본문을 자동으로 검증하는 파이프를 앱 전역에 건다. 한 번 걸어두면 모든 컨트롤러의 모든 DTO에 적용된다. 옵션 두 개가 중요하다.
 - `whitelist: true` — DTO에 선언하지 않은 속성은 본문에서 **잘라낸다**. 클라이언트가 `{ email, name, isAdmin: true }` 처럼 엉뚱한 필드를 끼워 보내도 `isAdmin`은 통째로 버려진다. 의도하지 않은 값이 서비스로 새어 드는 것을 막는다.
 - `transform: true` — 들어온 값을 DTO에 적힌 타입으로 **변환**한다. 뒤에서 다룰 `@IsInt()` `authorId`가 문자열 "1"로 와도 숫자 1로 바꿔준다.
- `app.listen(3000)` — 3000번 포트에서 요청을 받기 시작한다. 이 줄이 있어야 서버가 켜진다.

`bootstrap` 함수가 `async`인 이유는 `create`와 `listen`이 모두 비동기이기 때문이다. 마지막 줄에서 `bootstrap()`을 호출해 시동을 건다.

짚고 가기 — 검증 로직이 진입점에 단 한 줄(`useGlobalPipes`)뿐인 것을 눈여겨보자. Express에서는 핸들러마다 `if (!title) return res.status(400).`을 반복했다. 여기서는 "검증한다"는 선언만 전역에 한 번 하고, **규칙 자체는 DTO로 옮긴다**. 어디서 검증하느냐(파이프)와 무엇을 검증하느냐(DTO)가 깨끗이 갈린다.

루트 모듈 — 기능 모듈을 모은다

진입점이 출발점으로 삼은 `AppModule`이다.

```
import { Module } from "@nestjs/common";
import { UsersModule } from "../users/users.module";
import { PostsModule } from "../posts/posts.module";

// 루트 모듈 - 기능 모듈을 모아 앱을 구성한다.
// 이 장은 DB를 쓰지 않는다(인메모리). 11장에서 TypeORM, 12장에서 Prisma로 저장 계층을 붙인다.
@Module({
  imports: [UsersModule, PostsModule],
})
export class AppModule {}
```

`@Module({..})` 데코레이터가 이 클래스를 **모듈**로 표시한다. 안의 `imports` 배열은 "이 앱은 어떤 기능 모듈로 이루어지는가"를 적는 자리다. 여기서는 사용자 모듈과 게시글 모듈을 끌어와 앱을 구성한다.

루트 모듈에는 컨트롤러도 서비스도 없다. 실제 일은 기능 모듈들이 하고, 루트 모듈은 그것들을 모으는 역할만 한다. 기능이 늘어나면 새 모듈을 만들어 이 `imports` 배열에 한 줄 추가하면 된다. 앱이 모듈 단위로 커지는 구조다.

주석에 적혀 있듯, 지금은 `imports` 에 기능 모듈만 들어 있다. 11장에서는 여기에 `TypeOrmModule.forRoot(. )` 가, 12장에서는 Prisma 모듈이 더해진다. **DB 설정이 들어올 자리가 바로 이 루트 모듈이다.**

기능 모듈의 표준 형태 — 컨트롤러·서비스·DTO

이제 기능 모듈 하나를 통째로 본다. 사용자 모듈은 앞으로 만들 모든 기능 모듈의 본보기다. 컨트롤러(라우팅), 서비스(로직), DTO(검증), 모듈(조립) — 네 구성요소가 어떻게 맞물리는지 여기서 익히면 게시글 모듈은 같은 모양의 반복이다.

컨트롤러 — 요청을 받아 서비스에 넘긴다

```
import { Controller, Get, Post, Body, Param, Delete } from "@nestjs/common";
import { UserService } from "../users.service";
import { CreateUserDto } from "../dto/create-user.dto";

// @Controller("users") → 이 클래스의 라우트는 모두 /users 로 시작한다.
// 컨트롤러는 "요청을 받아 서비스에 위임"만 한다. 실제 로직은 서비스에.
@Controller("users")
export class UsersController {
  // 생성자 주입(DI) - NestJS가 UserService 인스턴스를 알아서 넣어준다.
  constructor(private readonly userService: UserService) {}

  @Post() // POST /users
  create(@Body() dto: CreateUserDto) {
    return this.userService.create(dto);
  }

  @Get() // GET /users
  findAll() {
    return this.userService.findAll();
  }

  @Get(":id") // GET /users/1
  findOne(@Param("id") id: string) {
    return this.userService.findOne(+id); // 경로 파라미터는 문자열 → +로 숫자 변환
  }

  @Delete(":id") // DELETE /users/1
  remove(@Param("id") id: string) {
    return this.userService.remove(+id);
  }
}
```

데코레이터가 라우트를 만든다.

- `@Controller("users")` — 이 클래스가 컨트롤러임을 표시하고, **공통 경로 접두사** `/users` 를 정한다. 클래스 안의 모든 메서드 라우트는 `/users` 로 시작한다.
- `@Post()`, `@Get()`, `@Delete()` — 메서드 위에 붙어 HTTP 메서드를 정한다. 괄호 안 인자는 접두사 뒤에 붙는 경로다. `@Get()` 은 `/users`, `@Get(":id")` 는 `/users/:id` 가 된다. Express의 `app.get("/users", ...)`, `app.get("/users/:id", ...)` 와 정확히 같은 라우트를, 호출이 아니라 **선언**으로 만든 것이다.

매개변수에도 데코레이터가 붙는다. Express에서 `req` 라는 큰 덩어리를 받아 `req.body`, `req.params.id` 를 꺼내 쓰던 일을, NestJS는 **필요한 조각만 콕 집어** 매개변수로 받는다.

- `@Body() dto: CreateUserDto` — 요청 본문을 통째로 받아 `dto` 에 담는다. Express의 `req.body` 에 대응한다. 단, 타입이 `CreateUserDto` 라는 점이 핵심이다. 전역 `ValidationPipe` 가 이 타입의 검증 규칙대로 본문을 검사한 뒤에 핸들러로 들어온다. 즉 핸들러가 받는 `dto` 는 이미 검증을 통과한 값이다.
- `@Param("id") id: string` — 경로 파라미터 `:id` 를 꺼낸다. `req.params.id` 에 대응한다. 경로 파라미터는 04장에서 봤듯 **항상 문자열**이므로, 서비스에 넘길 때 `+id` 로 숫자로 바꾼다. (`+` 는 문자열을 숫자로 바꾸는 단항 연산자다. `Number(id)` 와 같다.)

생성자 한 줄이 DI의 전부다.

- `constructor(private readonly usersService: UsersService) {}` — "이 컨트롤러는 `UsersService` 가 필요하다"는 선언이다. `new UsersService()` 는 어디에도 없다. NestJS가 인스턴스를 만들어 넣는다. `private readonly` 를 붙이면 그 인자가 곧바로 클래스의 속성(`this.usersService`)이 된다. 타입스크립트의 문법 설탕이지만, NestJS 코드에서 거의 관용구처럼 쓰인다.

메서드 본문은 모두 한 줄, `return this.usersService.무엇()` 이다. 컨트롤러가 **위임만 하고 로직을 두지 않는다**는 원칙이 그대로 드러난다. "컨트롤러는 얇게, 로직은 서비스로." 이 한 문장이 NestJS 구조의 절반이다.

짚고 가기 — 응답을 어떻게 보내는지 보이지 않는다. Express라면 `res.json(.)` 을 직접 불렀다. NestJS는 핸들러가 **return 한 값**을 알아서 JSON으로 직렬화해 응답한다. 상태코드도 기본값(GET 은 200, POST 는 201)이 알맞게 붙는다. `res` 객체를 다룰 일이 거의 없다.

서비스 — 진짜 일이 일어나는 곳

```
import {
  Injectable,
  ConflictException,
  NotFoundException,
} from "@nestjs/common";
import { CreateUserDto } from "../dto/create-user.dto";

// 도메인 모델 — DB가 없으니 그냥 평범한 객체 타입이다.
// (11장에서 TypeORM @Entity, 12장에서 Prisma model로 같은 모양을 정의하게 된다.)
export interface User {
  id: number;
  email: string;
  name: string;
  createdAt: Date;
}

@Injectable()
export class UsersService {
  // 인메모리 저장소 — 서버를 끄면 사라진다. 이 자리가 다음 장에서 DB로 바뀐다.
  private users: User[] = [];
  private nextId = 1;

  create(dto: CreateUserDto): User {
    if (this.users.some((u) => u.email === dto.email)) {
      throw new ConflictException("이미 존재하는 이메일입니다.");
    }
    const user: User = { id: this.nextId++, ...dto, createdAt: new Date() };
    this.users.push(user);
    return user;
  }

  findAll(): User[] {
    return this.users;
  }

  findOne(id: number): User {
    const user = this.users.find((u) => u.id === id);
    if (!user) throw new NotFoundException(`사용자 ${id}를 찾을 수 없습니다.`);
    return user;
  }

  remove(id: number): void {
    const index = this.users.findIndex((u) => u.id === id);
    if (index === -1) {
      throw new NotFoundException(`사용자 ${id}를 찾을 수 없습니다.`);
    }
    this.users.splice(index, 1);
  }
}
```



```

    }
  }
}

```

맨 위 `interface User` 는 사용자 한 명의 **모양**을 정의한 도메인 타입이다. DB가 없으니 평범한 객체 타입에 지나지 않는다. 주석에 적혀 있듯, 이 인터페이스가 11장에서는 TypeORM의 `@Entity` 클래스로, 12장에서는 Prisma 스키마의 `model` 로 모양을 바꾼다. 지금은 머릿속 약속이지만, 다음 장에서는 실제 DB 테이블의 설계도가 된다.

`@Injectable()` 데코레이터는 이 클래스가 **주입 가능한 서비스**임을 표시한다. 이 표식이 있어야 NestJS가 인스턴스를 관리하고, 컨트롤러나 다른 서비스의 생성자에 넣을 수 있다.

저장소는 메모리 배열이다.

- `private users: User[] = []` — 사용자 목록. `private nextId = 1` 은 새 사용자에게 줄 id다. Express 예제에서 모듈 최상단에 두던 `let posts = [].`, `let nextId` 와 같은 역할이되, **서비스 클래스 안에 갇혀** 있다. 바깥에서 이 배열을 직접 건드릴 수 없고, 오직 서비스의 메서드로만 접근한다. 이게 저장 계층을 다음 장에서 통째로 교체할 수 있는 이유다 — 사용자 배열을 DB 리포지토리로 바꿔도 컨트롤러는 아무것도 모른다.

각 메서드는 04장 CRUD와 논리가 같지만, 실패를 알리는 방식이 다르다.

- `create` — 이메일이 이미 있으면 `throw new ConflictException().` Express에서 `res.status(409).json().` 을 직접 보내던 것을, **예외를 던지는 것으로** 바꿨다. NestJS가 이 예외를 잡아 자동으로 **409 Conflict** 응답으로 변환한다. 통과하면 `{ id, ...dto, createdAt }` 객체를 만들어 배열에 넣고 반환한다. `...dto` 로 DTO의 `email · name` 이 펼쳐진다.
- `findOne / remove` — 못 찾으면 `throw new NotFoundException().` 이걸 **404 Not Found**로 변환된다. 서비스는 HTTP 상태코드를 직접 다루지 않는다. "이런 상황이다"라는 의미만 예외로 표현하고, 상태코드로 옮기는 일은 NestJS가 맡는다.

이 분리가 중요하다. 서비스는 `res` 도 `req` 도 모른다. HTTP를 모르는 순수한 로직이라, 나중에 테스트하거나 다른 곳에서 재사용하기 쉽다. 던지는 예외만 약속해 두면 컨트롤러는 손덜 게 없다.

NestJS의 HTTP 예외 — `NotFoundException (404)`, `ConflictException (409)`,

`BadRequestException (400)`, `UnauthorizedException (401)` 등이 `@nestjs/common` 에 갖춰져 있다. 상황에 맞는 예외를 `throw` 하면 알맞은 상태코드와 JSON 에러 본문이 자동으로 나간다. 04장에서 손으로 적던 `res.status(404).json({ message })` 가 `throw new NotFoundException(message)` 한 줄로 줄었다.

DTO — 본문의 모양과 검증 규칙을 한곳에

```
import { IsEmail, IsString, MinLength } from "class-validator";

// DTO(Data Transfer Object) - 요청 본문의 "모양"과 "검증 규칙"을 한곳에 정의.
// 전역 ValidationPipe가 이 데코레이터들을 읽어 자동으로 검증한다.
export class CreateUserDto {
  @IsEmail()
  email: string;

  @IsString()
  @MinLength(2)
  name: string;
}
```

DTO(Data Transfer Object) 는 "요청 본문이 어떤 모양이어야 하는가"를 적은 클래스다. 속성마다 `class-validator` 의 데코레이터로 **검증 규칙**을 붙인다.

- `@IsEmail()` — `email` 은 이메일 형식이어야 한다.
- `@IsString()` + `@MinLength(2)` — `name` 은 문자열이고 두 글자 이상이어야 한다.

규칙은 여기 선언만 되어 있다. 실제 검사는 진입점에서 전역으로 건 `ValidationPipe` 가 한다. 요청이 컨트롤러에 닿기 전, 파이프가 본문을 이 DTO에 비취보고 규칙을 어기면 **400 Bad Request**를 자동으로 돌려보낸다. `email` 이 이메일 형식이 아니거나 `name` 이 한 글자면, 핸들러는 아예 실행되지 않는다.

O4장의 검증과 비교하면 차이가 분명하다. Express에서는 핸들러 안에서 `if (!title || !content) return res.status(400).` 을 손으로 적었다. 검증 로직과 비즈니스 로직이 한 함수에 뒤섞였다. NestJS에서는 무엇을 검증할 지는 **DTO에**, 검증을 실행하는 일은 파이프에 떼어 놓는다. 그 결과 컨트롤러 핸들러에는 검증 코드가 한 줄도 없다.

`create(@Body() dto: CreateUserDto)` — 이 타입 표기 하나가 검증을 트리거한다.

모듈 — 묶고, 경계를 긋는다

```
import { Module } from "@nestjs/common";
import { UsersService } from "../users.service";
import { UsersController } from "../users.controller";

@Module({
  controllers: [UsersController],
  providers: [UsersService],
  // PostsModule이 UsersService를 주입받을 수 있도록 내보낸다(모듈 간 의존성).
  exports: [UsersService],
})
export class UsersModule {}
```

`@Module` 의 세 배열이 이 모듈의 경계를 정한다.

- `controllers: [UsersController]` — 이 모듈이 거느리는 컨트롤러. NestJS가 여기 적힌 컨트롤러의 라우트를 등록한다.
- `providers: [UsersService]` — 이 모듈 안에서 주입 가능한 서비스. `providers`에 등록해야 NestJS가 인스턴스를 만들어 `UserController`에 넣어준다. 등록하지 않으면 "주입할 게 없다"는 에러가 난다.
- `exports: [UsersService]` — `UsersService`를 **모듈 밖으로 내보낸다**. 이게 없으면 `UsersService`는 이 모듈 안에서만 쓸 수 있는 비공개 부품이다. 곧 볼 게시글 모듈이 `UsersService`를 빌려 쓰려면, 이쪽에서 먼저 `exports`로 문을 열어줘야 한다.

`providers`와 `exports`의 차이가 모듈 경계의 핵심이다. `providers`는 "이 모듈 안에서 쓴다", `exports`는 "이 모듈 밖에도 빌려준다"는 뜻이다. 기본은 비공개이고, 내보낼 것만 명시적으로 연다. 이 경계 덕분에 모듈끼리 무엇을 주고받는지 가 코드에 또렷이 드러난다.

모듈 간 협력 — DI의 진짜 맛

게시글 모듈은 사용자 모듈과 같은 네 구성요소를 갖는다. 컨트롤러·DTO·모듈 골격은 거의 판박이라, 다른 점만 짚는다. 핵심은 **게시글에 작성자가 있다**는 것 하나다. 글을 쓸 때 그 `authorId`가 실제 존재하는 사용자인지 확인해야 하고, 그러려면 게시글 모듈이 사용자 모듈의 서비스를 빌려 써야 한다.

DTO — 다른 모듈을 가리키는 외래 키

```
import { IsInt, IsString, MinLength } from "class-validator";

export class CreatePostDto {
  @IsString()
  @MinLength(1)
  title: string;

  @IsString()
  content: string;

  // 어떤 사용자가 쓴 글인지 - users의 id를 가리킨다.
  @IsInt()
  authorId: number;
}
```

`title`·`content`는 사용자 DTO와 같은 걸이다. 새로운 건 `@IsInt()` `authorId`다. 이 글을 누가 썼는지, 사용자의 `id`를 가리킨다. DB 용어로 하면 **외래 키(foreign key)**의 시초다. 지금은 그냥 숫자지만, 11장에서 테이블이 생기면 이 값이 실제 사용자 레코드를 가리키는 관계가 된다.

`@IsInt()`는 정수인지 검사한다. 앞서 진입점의 `transform: true` 덕분에, 클라이언트가 `authorId`를 문자열 "1"로 보내도 파이프가 숫자 1로 바꿔 검증한다.

서비스 — 다른 서비스를 주입받아 협력한다

```
import { Injectable, NotFoundException } from "@nestjs/common";
import { CreatePostDto } from "../dto/create-post.dto";
import { UsersService } from "../users/users.service";

export interface Post {
  id: number;
  title: string;
  content: string;
  authorId: number;
  createdAt: Date;
}

@Injectable()
export class PostsService {
  private posts: Post[] = [];
  private nextId = 1;

  // 다른 모듈의 서비스를 주입받는다 — 모듈 간 협력(DI)의 대표 예시.
  constructor(private readonly usersService: UsersService) {}

  create(dto: CreatePostDto): Post {
    // 작성자가 실제 존재하는지 확인 (없으면 UsersService가 404를 던진다).
    this.usersService.findOne(dto.authorId);
    const post: Post = { id: this.nextId++, ...dto, createdAt: new Date() };
    this.posts.push(post);
    return post;
  }

  findAll(): Post[] {
    return this.posts;
  }

  findOne(id: number): Post {
    const post = this.posts.find((p) => p.id === id);
    if (!post) throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
    return post;
  }

  remove(id: number): void {
    const index = this.posts.findIndex((p) => p.id === id);
    if (index === -1) {
      throw new NotFoundException(`게시글 ${id}를 찾을 수 없습니다.`);
    }
    this.posts.splice(index, 1);
  }
}
```

구조는 사용자 서비스와 같다. 결정적 차이는 생성자다.

- `constructor(private readonly userService: UsersService) {}` — 게시글 서비스가 사용자 서비스를 주입받는다. 컨트롤러가 서비스를 주입받던 것과 **완전히 같은 문법**이다. DI는 컨트롤러→서비스만이 아니라 서비스→서비스에도 똑같이 작동한다. 타입만 적으면 NestJS가 넣는다.

`create` 의 첫 줄이 협력의 실체다.

- `this.userService.findOne(dto.authorId)` — 글을 만들기 전에, 빌려온 사용자 서비스로 작성자가 실제 존재하는지 확인한다. 만약 없는 `authorId` 라면 사용자 서비스의 `findOne` 이 `NotFoundException` 을 던지고, 그게 그대로 **404 응답**이 된다. 게시글 서비스는 사용자 배열을 직접 들여다보지 않는다. 사용자에 관한 일은 사용자 서비스에 맡긴다. 각 서비스가 자기 데이터만 다루고, 필요할 때 서로 호출하는 것 — 이게 모듈 간 협력의 모범이다.

여기서 짚을 점이 하나 있다. 게시글 서비스와 사용자 서비스는 같은 **UsersService** 인스턴스를 공유한다. 사용자 컨트롤러로 만든 사용자를, 게시글 서비스가 그대로 조회할 수 있는 건 둘이 메모리 배열을 따로 갖는 게 아니라 한 인스턴스를 함께 쓰기 때문이다. NestJS의 서비스는 기본적으로 앱 전체에 하나(싱글턴)다. DI가 그 하나를 필요한 곳마다 똑같이 꽂아 준다.

모듈 — `imports` 로 다른 모듈을 끌어온다

```
import { Module } from "@nestjs/common";
import { PostsService } from "../posts.service";
import { PostsController } from "../posts.controller";
import { UsersModule } from "../users/users.module";

@Module({
  // UsersModule을 import해야 그 안에서 export한 UsersService를 주입받을 수 있다.
  imports: [UsersModule],
  controllers: [PostsController],
  providers: [PostsService],
})
export class PostsModule {}
```

`controllers` 와 `providers` 는 사용자 모듈과 같다. 새로운 건 `imports: [UsersModule]` 이다.

게시글 서비스가 사용자 서비스를 주입받으려면, NestJS가 어디서 **UserService** 를 가져올지 알아야 한다. 사용자 모듈이 그것을 `exports` 로 내보냈으니, 게시글 모듈은 사용자 모듈을 `imports` 로 끌어오면 된다. 그러면 사용자 모듈이 내보낸 **UserService** 가 게시글 모듈 안에서 주입 가능해진다.

두 모듈의 선언을 나란히 두면 짝이 보인다.

모듈	선언	뜻
사용자 모듈	<code>exports: [UserService]</code>	"이 서비스를 밖에 빌려준다"
게시글 모듈	<code>imports: [UsersModule]</code>	"그 모듈을 끌어와 쓴다"

`exports` (내보내는 쪽)와 `imports` (가져오는 쪽)가 한 쌍으로 맞물려야 모듈 간 의존성이 성립한다. 한쪽만 빠뜨리면 NestJS가 앱을 띄울 때 "주입할 게 없다"며 에러를 낸다. 이 명시적인 양쪽 선언이 NestJS의 모듈 경계다. 무엇을 누구에

게 빌려주고 누가 무엇을 빌려 오는지가 코드만 봐도 드러난다.

컨트롤러

```
@Controller("posts")
export class PostsController {
  constructor(private readonly postsService: PostsService) {}

  @Post() // POST /posts (body에 authorId 포함)
  create(@Body() dto: CreatePostDto) {
    return this.postsService.create(dto);
  }

  @Get() // GET /posts
  findAll() {
    return this.postsService.findAll();
  }

  @Get(":id") // GET /posts/1
  findOne(@Param("id") id: string) {
    return this.postsService.findOne(+id);
  }

  @Delete(":id") // DELETE /posts/1
  remove(@Param("id") id: string) {
    return this.postsService.remove(+id);
  }
}
```

사용자 컨트롤러와 한 글자 차이도 없을 만큼 똑같다. `@Controller("posts")` 로 접두사만 바뀌고, 게시글 서비스를 주입받아 위임한다. 컨트롤러가 이렇게까지 단순해지는 게 NestJS 구조의 의도다. 컨트롤러는 라우팅 파일 뿐, 일은 전부 서비스가 한다.

전체 흐름 — 요청 한 번이 거치는 길

게시글을 만드는 요청 하나를 따라가 보면 모든 조각이 어떻게 맞물리는지 보인다.

```

POST /posts {title, content, authorId}
    ↓
[ValidationPipe] — CreatePostDto 규칙대로 검증 (여기면 여기서 400)
    ↓
[PostsController.create(dto)] — 위임만
    ↓
[PostsService.create(dto)]
    |   └─ userService.findOne(authorId) — 작성자 확인 (없으면 404)
    └─ posts 배열에 추가
    ↓
return post — NestJS가 JSON + 201로 응답

```

검증(파이프) → 라우팅(컨트롤러) → 로직(서비스) → 협력(다른 서비스) → 응답(자동 직렬화). 각 단계가 다른 구성요소, 다른 역할로 나뉘어 있다. Express에서 한 핸들러 함수에 다 들어 있던 일들이 자리별로 흩어졌다. 처음엔 구성요소가 많아 번거롭게 느껴지지만, 규모가 커지면 "이 일은 어디서 하는가"를 헤맬 일이 없어진다. 그게 구조를 강제하는 프레임워크의 값어치다.

NestJS CLI — 골격은 직접 안 짤다

이 장의 구성요소들은 손으로 다 타이핑한 게 아니다. **NestJS CLI**가 골격을 만들어 준다. 설치하고 명령 하나면 모듈·컨트롤러·서비스·DTO가 한꺼번에 생긴다.

```

npm install -g @nestjs/cli      # 1회

nest new my-app                  # 새 프로젝트 생성
nest g resource users             # 모듈+컨트롤러+서비스+DTO 한 번에 (CRUD 골격)
nest g module posts              # 개별 생성도 가능
nest g controller posts
nest g service posts

```

`nest g resource users` 를 실행하면 모듈·컨트롤러·서비스와 DTO까지 CRUD 골격을 갖춰 만들어 준다.

`@Controller`, `@Injectable`, `@Module` 같은 데코레이터와 `findAll`, `findOne`, `create`, `remove` 메서드들이 이미 채워져 있다. 이 장의 사용자·게시글 모듈도 그렇게 생성한 골격에서 출발해, 서비스 내부를 **인메모리 배열**로 채운 것이다.

CLI가 골격을 정해진 모양으로 찍어내니, 누가 만들어도 폴더 구조와 파일 이름이 같다. 데코레이터를 빠뜨리거나 모듈 등록을 잊는 실수도 줄어든다. 이것 역시 "구조를 강제한다"의 한 모습이다.

Express와 NestJS — 같은 일을 어떻게 다르게 하나

04~09장의 Express와 이 장의 NestJS는 결국 같은 게시판을 만든다. 같은 일을 두 프레임워크가 어떻게 다르게 표현하는지 나란히 보면 NestJS의 성격이 또렷해진다.

하는 일	Express (04~09)	NestJS (10)
라우트 등록	<code>app.get("/users/:id", fn)</code> 호출	<code>@Get(":id")</code> 데코레이터 선언
본문 꺼내기	<code>req.body</code>	<code>@Body()</code> dto: <code>CreateUserDto</code>
경로 파라미터	<code>req.params.id</code>	<code>@Param("id") id: string</code>
응답	<code>res.status(201).json(data)</code>	<code>return data</code> (자동 직렬화)
검증	핸들러 안 <code>if</code> 문	DTO 데코레이터 + 전역 파이프
의존성 마련	<code>const svc = new Service()</code>	생성자에 타입만 — DI
로직 위치	핸들러 안에 자유롭게	서비스로 강제 분리
에러 응답	<code>res.status(404).json(.)</code>	<code>throw new NotFoundException()</code>

Express는 "원하는 대로 짜라"고 자유를 준다. NestJS는 "이 자리엔 이걸 뒤라"고 구조를 강제한다. 작은 서버라면 Express의 가벼움이 빠르고, 사람이 늘고 코드가 커지는 서비스라면 NestJS의 일관성이 산다. 둘 중 무엇이 옳다기보다, 둘의 성격을 알고 골라 쓰는 것이다.

흔한 실수

- 서비스를 **providers**에 등록하지 않는다. → "Nest can't resolve dependencies" 에러. 주입하려면 그 서비스가 모듈의 **providers**에 있어야 한다.
- **exports** 없이 다른 모듈에서 쓰려 한다. → 사용자 모듈이 `UserService`를 **exports**하지 않으면, 게시글 모듈이 **imports**해도 주입받지 못한다. 내보내는 쪽과 가져오는 쪽 양쪽 선언이 다 필요하다.
- **@Injectable()**을 빠뜨린다. → 데코레이터가 없으면 NestJS가 그 클래스를 서비스로 인식하지 못해 주입이 깨진다.
- 진입점에서 **ValidationPipe**를 안 건다. → DTO에 `@IsEmail()` 등을 적어도 아무 검증도 일어나지 않는다. 규칙(DTO)과 실행(파이프)은 둘 다 있어야 한다.
- 경로 파라미터를 문자열인 채 서비스에 넘긴다. → `@Param("id") id`는 `"1"`(문자열). `+id`로 바꾸지 않으면 `find`가 숫자 `id`와 비교에 실패한다. Express에서 `Number(req.params.id)`를 빠뜨린 것과 같은 실수다.

- 서비스에서 `res` 로 응답하려 한다. → NestJS 서비스는 HTTP를 모른다. 값을 `return` 하고, 실패는 예외를 `throw` 한다.

직접 해보기

1. 사용자에게 수정 기능을 더해 보자. `UpdateUserDto` 를 만들고(전부 선택 입력이면 `@IsOptional()` 을 붙인다), 컨트롤러에 `@Patch("/:id")` , 서비스에 `update(id, dto)` 를 추가한다. `@Patch` 는 `@nestjs/common` 에서 `@Get` 과 같은 자리에 있다.
2. 게시글 서비스의 `findAll` 을 **작성자 정보까지** 담아 돌려주게 바꿔 보자. 각 글에 `this.userService.findOne(post.authorId)` 로 작성자를 찾아 `{..post, author }` 형태로 합친다. (존재하지 않는 작성자가 있을 수 있으니 예외 처리도 생각해 보자.)
3. `CreatePostDto` 의 `content` 에 최소 길이 규칙 `@MinLength(5)` 를 더하고, 네 글자 본문으로 `POST /posts` 를 보내 **400 응답과 에러 메시지**가 어떻게 생겼는지 확인해 보자. 검증이 핸들러에 닿기 전에 막히는 것을 눈으로 본다.

정리

- NestJS 코드는 **컨트롤러(요청)·서비스(로직)·모듈(조립)** 세 자리로 나뉜다. 컨트롤러는 얇게 위임만 하고, 로직은 서비스로 강제 분리된다.
- **데코레이터**가 그 자리를 표시한다. `@Controller` · `@Injectable` · `@Module` 로 역할을, `@Get` · `@Post` · `@Body` · `@Param` 으로 라우트와 입력을 선언한다.
- **DI**는 "직접 `new` 하지 않고 생성자에 타입만 적으면 NestJS가 넣는다". 컨트롤러→서비스, 서비스→서비스 모두 같은 문법이다.
- 모듈 경계는 **exports (내보내기)**와 **imports (가져오기)** 한 쌍으로 그어진다. 이 명시적 선언이 모듈 간 의존성을 코드에 명시한다.
- 검증은 **DTO에 선언하고 전역 `ValidationPipe` 가 실행한다**. 컨트롤러에서 검증 코드가 사라진다.
- 저장은 아직 메모리 배열이다. 서비스 안에 갇혀 있어, 다음 장에서 이 부분만 DB로 교체하면 된다.

다음 장(**11 · TypeORM**)에서는 이 인메모리 저장소를 PostgreSQL과 TypeORM 리포지토리로 교체한다. `interface User` 는 `@Entity` 클래스가 되고, 사용자 배열은 진짜 테이블이 된다. 컨트롤러와 DTO는 거의 그대로 둔 채 **저장 계층만 바꾸는** 감각을 여기서 처음 맞본다.